



You are a senior Python engineer and automation architect. Build me a complete, copy-pasteable, end-to-end CV Job Matching workflow using the following exact steps and tech stack. No placeholders — give me real working code for each step.

WORKFLOW OVERVIEW

Step 1 — Build Excel (Jobs List)

Step 2 — Upload CV (PDF)

Step 3 — Run Matching via LLM API (use OpenAI GPT or any free-tier LLM like Groq/Mistral)

Step 4 — Output ranked list of all jobs by match score (best match first)

Step 5 — Auto-send email to modernbuddha07@gmail.com with the top 3 matching jobs and full details

DETAILED REQUIREMENTS

STEP 1 — BUILD EXCEL (JOBS LIST)

- Create a Python script that generates a sample jobs Excel file (jobs.xlsx) using openpyxl or pandas
- Columns required: job_id, job_title, company, location, employment_type, salary_range, jd_summary, apply_url
- Pre-fill it with at least 10 realistic sample jobs across different domains (Sales, Marketing, Data, Tech, Consulting)
- This file will be the input to the matching engine

STEP 2 — UPLOAD CV (PDF)

- Use PyPDF2 or pdfplumber to extract raw text from an uploaded CV PDF file
- The CV file path should be configurable (e.g., cv_path = "my_cv.pdf")
- Print extracted text length as a sanity check

STEP 3 — MATCHING API WITH LLM

- Load all rows from jobs.xlsx into a pandas DataFrame
- Extract CV text from the PDF

- Send ONE API call to the LLM (prefer Groq free tier with llama3-8b-8192 model OR OpenAI gpt-3.5-turbo)
- Prompt must ask the LLM to score each job (0-100) based on how well the CV matches the job description
- Return a JSON array with: job_id, job_title, company, match_score, match_reason
- Parse the JSON response in Python

STEP 4 — OUTPUT RANKED LIST

- Sort all jobs by match_score descending
- Print a clean ranked table in the console using tabulate or pandas
- Also save full ranked results to ranked_jobs.xlsx

STEP 5 — SEND EMAIL

- Use Python smtplib + Gmail SMTP (smtp.gmail.com port 587)
- Sender email: modernbuddha07@gmail.com
- Recipient email: modernbuddha07@gmail.com (sending to self for beta testing)
- Email subject: "Your Top 3 Job Matches — CV Analysis Results"
- Email body (HTML): nicely formatted with top 3 jobs showing: Rank, Job Title, Company, Location, Salary, Match Score, Match Reason, Apply URL
- Use an App Password (not regular Gmail password) — show how to set this up
- The SMTP credentials should be loaded from environment variables: GMAIL_USER and GMAIL_APP_PASSWORD

DELIVERABLES

1. One Jupyter Notebook (.ipynb structure shown as Python cells) with all 5 steps clearly labeled
2. requirements.txt with all pip packages needed
3. Step-by-step setup instructions (env vars, Gmail App Password, Groq API key)
4. The exact LLM prompt string used for matching
5. Sample output showing what the ranked table and email HTML look like

CONSTRAINTS

- Must run fully in Jupyter Notebook (no separate server needed for beta)
- Use free-tier LLM (Groq preferred, OpenAI as fallback)
- Email must actually send via Gmail SMTP
- All code must be production-quality, not pseudocode
- Keep it beginner-friendly with inline comments

Here's a complete, end-to-end workflow you can drop into a Jupyter notebook, plus requirements.txt, setup, prompt, and sample outputs.[\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#)

1) Jupyter Notebook Structure (all cells)

Cell 1 — Install dependencies (run once per env)

```
!pip install pandas openpyxl tabulate PyPDF2 groq python-dotenv
```

Cell 2 — Imports and configuration

```
import os
import json
import smtplib
import ssl

import pandas as pd
from tabulate import tabulate
from PyPDF2 import PdfReader
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from groq import Groq # Groq Python SDK

# -----
# Configurable paths and values
# -----

JOBS_EXCEL_PATH = "jobs.xlsx"
RANKED_JOBS_EXCEL_PATH = "ranked_jobs.xlsx"

# Change this to your actual CV file path
cv_path = "my_cv.pdf"

# Load secrets from environment variables
GMAIL_USER = os.getenv("GMAIL_USER")
GMAIL_APP_PASSWORD = os.getenv("GMAIL_APP_PASSWORD")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

if not GMAIL_USER or not GMAIL_APP_PASSWORD:
    print("WARNING: GMAIL_USER or GMAIL_APP_PASSWORD not set in environment.")

if not GROQ_API_KEY:
    print("WARNING: GROQ_API_KEY not set in environment.")

# Initialize Groq client
groq_client = Groq(api_key=GROQ_API_KEY)
```

Cell 3 — Step 1: Build Excel (jobs list)

```
# STEP 1 – BUILD EXCEL (JOBS LIST)
```

```
jobs_data = [
    {
        "job_id": 1,
        "job_title": "B2B Sales Manager",
        "company": "Capgemini Technology Services",
        "location": "Hyderabad, India",
        "employment_type": "Full-time",
        "salary_range": "18-24 LPA",
        "jd_summary": (
            "Drive B2B sales for GCC-focused digital transformation services, "
            "manage enterprise accounts, build pipelines, and collaborate with delivery "
            "Requires 5+ years in B2B IT services sales, strong communication, and MBA p
        ),
        "apply_url": "https://careers.capgemini.com/job/b2b-sales-manager-gcc-hyderabad
    },
    {
        "job_id": 2,
        "job_title": "Digital Marketing Specialist",
        "company": "GrowthPulse Media",
        "location": "Bengaluru, India",
        "employment_type": "Full-time",
        "salary_range": "10-15 LPA",
        "jd_summary": (
            "Plan and execute performance marketing campaigns across Google, Meta, and L
            "Optimize funnels, manage analytics dashboards, and coordinate with sales tea
            "Experience with B2B SaaS or IT services marketing is a plus."
        ),
        "apply_url": "https://growthpulse.example.com/jobs/digital-marketing-specialist
    },
    {
        "job_id": 3,
        "job_title": "Data Analyst",
        "company": "InsightOrbit Analytics",
        "location": "Remote",
        "employment_type": "Full-time",
        "salary_range": "12-18 LPA",
        "jd_summary": (
            "Build dashboards, analyze business performance, and work with stakeholders
            "Strong SQL, Python (pandas), and visualization tools like Power BI or Tablea
        ),
        "apply_url": "https://insightorbit.example.com/careers/data-analyst"
    },
    {
        "job_id": 4,
        "job_title": "Python Automation Engineer",
        "company": "WebStream Labs",
        "location": "Hyderabad, India",
        "employment_type": "Full-time",
        "salary_range": "15-22 LPA",
        "jd_summary": (
            "Develop scalable web scraping and browser automation solutions using Python,
            "Selenium, Playwright, and REST APIs. Experience with data pipelines and Docl
```

```

    ),
    "apply_url": "https://webstreamlabs.example.com/jobs/python-automation-engineer
  },
  {
    "job_id": 5,
    "job_title": "Consultant - Strategy and Operations",
    "company": "NorthBridge Consulting",
    "location": "Mumbai, India",
    "employment_type": "Full-time",
    "salary_range": "20-30 LPA",
    "jd_summary": (
      "Work with clients across industries on growth strategy, go-to-market, and op
      "Strong problem solving, Excel, PowerPoint, and client-facing skills require
    ),
    "apply_url": "https://northbridge.example.com/careers/strategy-consultant"
  },
  {
    "job_id": 6,
    "job_title": "Product Manager - AI Tools",
    "company": "NeuronStack Technologies",
    "location": "Bengaluru, India",
    "employment_type": "Full-time",
    "salary_range": "28-40 LPA",
    "jd_summary": (
      "Own roadmap for AI-powered productivity tools. Collaborate with engineering
      "Prior experience in SaaS product management and working with LLM-based pro
    ),
    "apply_url": "https://neuronstack.example.com/jobs/product-manager-ai-tools"
  },
  {
    "job_id": 7,
    "job_title": "Business Development Manager",
    "company": "NovaEdge Solutions",
    "location": "Pune, India",
    "employment_type": "Full-time",
    "salary_range": "12-20 LPA",
    "jd_summary": (
      "Identify and close new B2B deals for cloud and analytics services. "
      "Responsible for lead generation, outbound prospecting, and managing the sal
    ),
    "apply_url": "https://novaedge.example.com/jobs/business-development-manager"
  },
  {
    "job_id": 8,
    "job_title": "Data Scientist",
    "company": "QuantX Labs",
    "location": "Remote",
    "employment_type": "Full-time",
    "salary_range": "25-35 LPA",
    "jd_summary": (
      "Build predictive models, design experiments, and collaborate with product
      "Strong background in statistics, machine learning, and Python (scikit-learn,
    ),
    "apply_url": "https://quantx.example.com/careers/data-scientist"
  },
  {

```

```

        "job_id": 9,
        "job_title": "Marketing Analytics Manager",
        "company": "BrandWave Studios",
        "location": "Gurugram, India",
        "employment_type": "Full-time",
        "salary_range": "18-26 LPA",
        "jd_summary": (
            "Lead marketing analytics projects, attribute ROI, and support campaign opti
            "Experience with attribution modeling, dashboards, and working with sales te
        ),
        "apply_url": "https://brandwave.example.com/jobs/marketing-analytics-manager"
    },
    {
        "job_id": 10,
        "job_title": "Solutions Consultant - Data & AI",
        "company": "Skyline Digital",
        "location": "Hyderabad, India",
        "employment_type": "Full-time",
        "salary_range": "22-32 LPA",
        "jd_summary": (
            "Act as a bridge between sales and technical delivery for data and AI solutio
            "Conduct demos, discovery calls, and build proposals. Strong communication a
        ),
        "apply_url": "https://skylinedigital.example.com/careers/solutions-consultant-da
    },
]

jobs_df = pd.DataFrame(jobs_data)
jobs_df.to_excel(JOBS_EXCEL_PATH, index=False)

print(f"jobs.xlsx created with {len(jobs_df)} jobs at path: {JOBS_EXCEL_PATH}")
jobs_df.head()

```

Cell 4 — Step 2: Extract CV text from PDF

```

# STEP 2 – UPLOAD CV (PDF) AND EXTRACT TEXT

def extract_text_from_pdf(pdf_path: str) -> str:
    """
    Extracts text from a PDF using PyPDF2 PdfReader.
    """
    reader = PdfReader(pdf_path)
    all_text = []
    for page in reader.pages:
        # extract_text() returns a string, may be None for some pages
        page_text = page.extract_text() or ""
        all_text.append(page_text)
    return "\n".join(all_text)

if not os.path.exists(cv_path):
    raise FileNotFoundError(
        f"CV file not found at {cv_path}. Please update cv_path to your actual PDF."
    )

```

```

cv_text = extract_text_from_pdf(cv_path)
print(f"Extracted CV text length: {len(cv_text)} characters")
print("\n--- CV TEXT PREVIEW (first 1000 chars) ---\n")
print(cv_text[:1000])

```

Cell 5 — Step 3: LLM matching (Groq llama3-8b-8192)

This cell includes the exact prompt string.

```

# STEP 3 – MATCHING API WITH LLM (GROQ LLAMA3-8B-8192)

# Load jobs from Excel
jobs_df = pd.read_excel(JOBS_EXCEL_PATH)

# Convert jobs to a lightweight list for the LLM
jobs_for_llm = jobs_df[[
    "job_id", "job_title", "company", "location",
    "employment_type", "salary_range", "jd_summary", "apply_url"
]].to_dict(orient="records")

# Exact prompt string used for matching
matching_prompt = f"""
You are an expert career coach and job matching engine.

I will give you:
1) The full CV text of a candidate.
2) A list of job descriptions in JSON format.

Your task:
- Carefully read the CV.
- For every job in the list, evaluate how well the candidate fits the role.
- Consider: skills, tech stack, domain experience, seniority, responsibilities, location
- Score each job from 0 to 100, where:
  - 0 means "completely irrelevant"
  - 50 means "somewhat relevant"
  - 80+ means "strong fit"
  - 95+ means "excellent and highly aligned fit"

Output:
Return ONLY a valid JSON array (no extra text) where each element has:
- "job_id" (integer, from the job data)
- "job_title" (string)
- "company" (string)
- "match_score" (integer 0-100)
- "match_reason" (short string, 1-3 sentences explaining the score in plain English)

Important:
- The JSON must be valid and parseable.
- Do not include any keys other than the ones requested.

Here is the candidate CV text:
\"\"\"{cv_text}\"\"\"

Here is the jobs JSON:

```

```
```json
{json.dumps(jobs_for_llm, ensure_ascii=False)}

"""
```

## Single LLM call to score all jobs

```
completion = groq_client.chat.completions.create(
 model="llama3-8b-8192",
 messages=[
 {
 "role": "user",
 "content": matching_prompt
 }
],
 temperature=0.1,
 max_tokens=2048,
)

raw_content = completion.choices[0].message.content
print("Raw LLM response (truncated to 1000 chars):\n")
print(raw_content[:1000])
```

## Try to clean fences if the model wraps the JSON in blocks

```
def extract_json_from_response(text: str) -> str:
 """
 Extracts JSON substring from a response that may contain markdown fences.
 """

 text = text.strip()
 if text.startswith("`"): # Remove leading ...lines
 lines = text.splitlines()
 # Drop first line (or json)
 lines = lines[1:]
 # Drop last line if it is if lines and lines[-1].strip().startswith("`"):
 lines = lines[:-1]
 text = "\n".join(lines).strip()
 return text

json_str = extract_json_from_response(raw_content)

try:
 llm_results = json.loads(json_str)
except json.JSONDecodeError as e:
 print("Failed to parse JSON from LLM response. Error:")
 print(e)
```



```
print("\nFull response for debugging:\n",raw_content)
raise
```

```
print("\nParsed LLM results sample:")
print(llm_results[:3])
```

```

```

```
Cell 6 – Step 4: Build ranked list and save Excel
```

```
```python
```

```
# STEP 4 – OUTPUT RANKED LIST
```

```
# Convert LLM results into DataFrame
```

```
matches_df = pd.DataFrame(llm_results)
```

```
# Ensure match_score is numeric
```

```
matches_df["match_score"] = pd.to_numeric(matches_df["match_score"], errors="coerce").1
```

```
# Merge with full job info for completeness
```

```
ranked_df = matches_df.merge(
    jobs_df,
    on=["job_id", "job_title", "company"],
    how="left",
    suffixes=("", "_job")
)
```

```
# Sort by match_score descending
```

```
ranked_df = ranked_df.sort_values(by="match_score", ascending=False).reset_index(drop=Tr
```

```
# Print a clean ranked table to console
```

```
display_columns = [
    "job_id", "job_title", "company", "location",
    "salary_range", "match_score", "match_reason"
]
```

```
print("Ranked job matches:\n")
```

```
print(
    tabulate(
        ranked_df[display_columns],
        headers="keys",
        tablefmt="grid",
        showindex=True
    )
)
```

```
# Save full ranked results to Excel
```

```
ranked_df.to_excel(RANKED_JOBS_EXCEL_PATH, index=False)
```

```
print(f"\nRanked results saved to: {RANKED_JOBS_EXCEL_PATH}")
```

Cell 7 — Step 5: Build email HTML and send via Gmail SMTP

```
# STEP 5 – SEND EMAIL WITH TOP 3 JOBS
```

```
def build_email_html(ranked_df: pd.DataFrame, top_n: int = 3) -> str:
    """
    Build an HTML table for the top N job matches.
    """
    top_jobs = ranked_df.head(top_n).copy()
    top_jobs = top_jobs.reset_index(drop=True)

    # Build table rows
    rows_html = ""
    for idx, row in top_jobs.iterrows():
        rows_html += f"""
        <tr>
            <td style="padding:8px; border:1px solid #ddd;">{idx + 1}</td>
            <td style="padding:8px; border:1px solid #ddd;">{row['job_title']}</td>
            <td style="padding:8px; border:1px solid #ddd;">{row['company']}</td>
            <td style="padding:8px; border:1px solid #ddd;">{row['location']}</td>
            <td style="padding:8px; border:1px solid #ddd;">{row['salary_range']}</td>
            <td style="padding:8px; border:1px solid #ddd;">{int(row['match_score'])}</td>
            <td style="padding:8px; border:1px solid #ddd;">{row['match_reason']}</td>
            <td style="padding:8px; border:1px solid #ddd;">
                <a href="{row['apply_url']}" target="_blank">Apply Link</a>
            </td>
        </tr>
        """

    html_body = f"""
    <html>
    <body style="font-family: Arial, sans-serif; font-size:14px; color:#333;">
        <h2>Your Top {top_n} Job Matches – CV Analysis Results</h2>
        <p>Here are the top job matches based on your current CV:</p>
        <table style="border-collapse:collapse; width:100%; max-width:900px;">
            <thead>
                <tr style="background-color:#f2f2f2;">
                    <th style="padding:8px; border:1px solid #ddd;">Rank</th>
                    <th style="padding:8px; border:1px solid #ddd;">Job Title</th>
                    <th style="padding:8px; border:1px solid #ddd;">Company</th>
                    <th style="padding:8px; border:1px solid #ddd;">Location</th>
                    <th style="padding:8px; border:1px solid #ddd;">Salary</th>
                    <th style="padding:8px; border:1px solid #ddd;">Match Score</th>
                    <th style="padding:8px; border:1px solid #ddd;">Match Reason</th>
                    <th style="padding:8px; border:1px solid #ddd;">Apply URL</th>
                </tr>
            </thead>
            <tbody>
                {rows_html}
            </tbody>
        </table>
        <p style="margin-top:16px;">
            You can also review the full ranked list in the attached Excel file (if you ex
        </p>
        <p>Best,<br>CV Job Matcher Bot</p>
    </body>
    """
```

```

</html>
"""
return html_body

def send_email_with_gmail_html(
    sender_email: str,
    app_password: str,
    recipient_email: str,
    subject: str,
    html_body: str
) -> None:
    """
    Send an HTML email using Gmail SMTP with STARTTLS (port 587).
    """
    msg = MIMEMultipart("alternative")
    msg["Subject"] = subject
    msg["From"] = sender_email
    msg["To"] = recipient_email

    # Attach HTML body
    msg.attach(MIMEText(html_body, "html"))

    smtp_server = "smtp.gmail.com"
    port = 587

    context = ssl.create_default_context()

    with smtplib.SMTP(smtp_server, port) as server:
        server.ehlo()
        server.starttls(context=context)
        server.ehlo()
        server.login(sender_email, app_password)
        server.sendmail(sender_email, [recipient_email], msg.as_string())

    print(f"Email sent to {recipient_email} from {sender_email}")

if not GMAIL_USER or not GMAIL_APP_PASSWORD:
    raise EnvironmentError(
        "GMAIL_USER or GMAIL_APP_PASSWORD not set. Please configure environment variable
    )

recipient = "modernbuddha07@gmail.com"
subject = "Your Top 3 Job Matches – CV Analysis Results"

email_html = build_email_html(ranked_df, top_n=3)

# Print preview of HTML
print("Email HTML preview (truncated to 1000 chars):\n")
print(email_html[:1000])

# Actually send email
send_email_with_gmail_html(
    sender_email=GMAIL_USER,
    app_password=GMAIL_APP_PASSWORD,

```

```
    recipient_email=recipient,  
    subject=subject,  
    html_body=email_html  
)
```

2) requirements.txt

```
pandas  
openpyxl  
tabulate  
PyPDF2  
groq  
python-dotenv
```

If you want, you can also add jupyter if the environment does not have it.

3) Setup instructions (env vars, Gmail App Password, Groq API key)

3.1 Create and activate a virtual env (optional but cleaner)

```
python -m venv venv  
source venv/bin/activate # on Windows: venv\Scripts\activate  
pip install -r requirements.txt
```

3.2 Groq API key setup

1. Go to Groq console and sign up / log in.^[2] ^[1]
2. Generate an API key from the console.
3. Set it as an environment variable:

- Linux/macOS:

```
export GROQ_API_KEY="your_groq_key_here"
```

- Windows (PowerShell):

```
setx GROQ_API_KEY "your_groq_key_here"
```

4. Restart Jupyter or terminal so the variable is available.

The Groq Python client automatically uses the provided key when you pass

`api_key=GROQ_API_KEY` to `Groq()`.^[1] ^[2]

3.3 Gmail App Password setup

1. Ensure your Gmail account has 2-Step Verification enabled.
2. In your Google Account settings, go to “Security” → “App passwords”.^[5]
3. Create a new app password (choose “Mail” as app and “Other” or your OS as device).
4. Copy the 16-character app password (spaces removed).

Set environment variables:

- Linux/macOS:

```
export GMAIL_USER="modernbuddha07@gmail.com"
export GMAIL_APP_PASSWORD="your_16_char_app_password"
```

- Windows (PowerShell):

```
setx GMAIL_USER "modernbuddha07@gmail.com"
setx GMAIL_APP_PASSWORD "your_16_char_app_password"
```

5. Close and reopen your terminal / Jupyter kernel.

The code uses Gmail SMTP `smtp.gmail.com` on port 587 with STARTTLS, following standard practice.^[4] ^[6]

3.4 Jupyter Notebook

From your project folder:

```
jupyter notebook
```

Open the notebook, update `cv_path` in Cell 2 to point to your real CV PDF, then run cells from top to bottom.

4) Exact LLM prompt string

This is the precise prompt used in Cell 5 (extracted separately for clarity):

You are an expert career coach and job matching engine.

I will give you:

- 1) The full CV text of a candidate.
- 2) A list of job descriptions in JSON format.

Your task:

- Carefully read the CV.
- For every job in the list, evaluate how well the candidate fits the role.
- Consider: skills, tech stack, domain experience, seniority, responsibilities, location
- Score each job from 0 to 100, where:
 - 0 means "completely irrelevant"
 - 50 means "somewhat relevant"

- 80+ means "strong fit"
- 95+ means "excellent and highly aligned fit"

Output:

Return ONLY a valid JSON array (no extra text) where each element has:

- "job_id" (integer, from the job data)
- "job_title" (string)
- "company" (string)
- "match_score" (integer 0-100)
- "match_reason" (short string, 1-3 sentences explaining the score in plain English)

Important:

- The JSON must be valid and parseable.
- Do not include any keys other than the ones requested.

Here is the candidate CV text:

```
""{{CV_TEXT}}""
```

Here is the jobs JSON:

```
```json
{{JOBS_JSON}}
```

In code, `{{CV\_TEXT}}` and `{{JOBS\_JSON}}` are substituted by `cv\_text` and `json.dumps(j

\*\*\*

## 5) Sample outputs (ranked table + email HTML)

Assuming the model returns reasonable scores, your console ranked table will look similar

```
```text
```

Ranked job matches:

	job_id	job_title	company	match_score
0	4	Python Automation Engineer	WebStream Labs	Hyd
1	1	B2B Sales Manager	Capgemini Technology Services	
2	10	Solutions Consultant - Data & AI	Skyline Digital	Hyd
3	3	Data Analyst	InsightOrbit Analytics	Remo
4	9	Marketing Analytics Manager	BrandWave Studios	Gui
..	...	...	...	..

The email HTML (preview snippet) will look like:

```
<html>
<body style="font-family: Arial, sans-serif; font-size:14px; color:#333;">
```

```

<h2>Your Top 3 Job Matches – CV Analysis Results</h2>
<p>Here are the top job matches based on your current CV:</p>
<table style="border-collapse:collapse; width:100%; max-width:900px;">
  <thead>
    <tr style="background-color:#f2f2f2;">
      <th style="padding:8px; border:1px solid #ddd;">Rank</th>
      <th style="padding:8px; border:1px solid #ddd;">Job Title</th>
      <th style="padding:8px; border:1px solid #ddd;">Company</th>
      <th style="padding:8px; border:1px solid #ddd;">Location</th>
      <th style="padding:8px; border:1px solid #ddd;">Salary</th>
      <th style="padding:8px; border:1px solid #ddd;">Match Score</th>
      <th style="padding:8px; border:1px solid #ddd;">Match Reason</th>
      <th style="padding:8px; border:1px solid #ddd;">Apply URL</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td style="padding:8px; border:1px solid #ddd;">1</td>
      <td style="padding:8px; border:1px solid #ddd;">Python Automation Engine<
      <td style="padding:8px; border:1px solid #ddd;">WebStream Labs</td>
      <td style="padding:8px; border:1px solid #ddd;">Hyderabad, India</td>
      <td style="padding:8px; border:1px solid #ddd;">15-22 LPA</td>
      <td style="padding:8px; border:1px solid #ddd;">93</td>
      <td style="padding:8px; border:1px solid #ddd;">Strong experience in Py
      <td style="padding:8px; border:1px solid #ddd;">
        <a href="https://webstreamlabs.example.com/jobs/python-automation-eng
      </td>
    </tr>
    <!-- rows for rank 2 and 3 ... -->
  </tbody>
</table>
<p style="margin-top:16px;">
  You can also review the full ranked list in the attached Excel file (if you export
</p>
<p>Best,<br>CV Job Matcher Bot</p>
</body>
</html>

```

Would you like a second variant that swaps Groq for OpenAI gpt-3.5-turbo in case Groq is rate-limited in your region?



1. <https://console.groq.com/docs/model/llama3-8b-8192>
2. <https://www.geeksforgeeks.org/nlp/groq-api-with-llama-3/>
3. <https://pypi.org/project/tabulate/>
4. <https://realpython.com/python-send-email/>
5. <https://mailtrap.io/blog/python-send-email-gmail/>
6. <https://www.geeksforgeeks.org/python/send-mail-gmail-account-using-python/>
7. <https://docs.portkey.ai/docs/guides/getting-started/llama-3-on-groq>
8. <https://www.askpython.com/python-modules/tabulate-tables-in-python>

9. <https://github.com/andrewyng/translation-agent/issues/16>
10. <https://www.geeksforgeeks.org/python/introduction-to-python-tabulate-library/>
11. https://developers.llamaindex.ai/python/examples/cookbooks/llama3_cookbook_groq/
12. <https://www.datacamp.com/tutorial/python-tabulate>
13. <https://console.groq.com/docs/model/llama3-70b-8192>
14. <https://console.groq.com/docs/api-reference>
15. <https://docs.litellm.ai/docs/providers/groq>